

高级语言程序设计 2-2 项目报告

南开大学 工科试验班

姓名：刘宏轩

学号：2511205

班级：模拟 2-1

2026 年 5 月 7 日

目录

1	作业题目	2
2	开发软件	2
3	课题要求	2
4	主要流程	2
4.1	整体流程	2
4.2	算法或公式	2
4.3	单元测试（设计思路）	2
5	单元测试	3
5.1	测试案例定义	3
5.2	测试结果	3
6	收获	3
6.1	信号与槽机制的应用	3
6.2	自定义控件与列表嵌入	4
6.3	资源管理体会	4
7	AI 工具使用披露	4

1 作业题目

番茄钟 (TomatoClock) —— 基于 Qt、C++ 的待办管理与专注计时应用。

2 开发软件

- Qt 6.11.0
- CMake 3.30.5
- 编译器: MinGW 13.1.0
- IDE: Qt Creator 19.0.1

3 课题要求

1. 面向对象: 采用类的封装、继承、多态进行设计。
2. 图形用户界面: 使用 Qt Widgets 构建完整交互界面。
3. 多媒体支持: 集成音乐播放、全屏专注模式。
4. 数据可视化: 通过 Qt Charts 展示专注时长统计。
5. 待办事项管理: 支持添加、删除、完成待办, 并关联专注计时。

4 主要流程

4.1 整体流程

应用围绕番茄工作法设计, 主要类及关系如下:

- **MainWindow**: 主窗口, 管理待办列表、总计时和图表;
- **FocusWindow**: 全屏专注倒计时窗口, 能够暂停、提前结束、播放音乐;
- **AddTodoDialog**: 添加待办对话框;
- **TodoItemWidget**: 待办事项控件;

用户通过主界面添加待办 (名称、时长、备注), 点击“开始”后进入全屏专注模式, **FocusWindow** 使用 **QTimer** 每秒更新倒计时, 并通过 **QMediaPlayer** 播放背景音乐。专注结束时, **FocusWindow** 发射信号通知 **MainWindow**, 后者累计总时长、更新待办完成状态并刷新图表。同时提供不依赖添加待办的快速开始功能。

4.2 算法或公式

- 倒计时: $\text{分钟} = \text{剩余秒数} / 60$, $\text{秒} = \text{剩余秒数} \% 60$ 。
- 总专注分钟数 = 每次专注完成的秒数之和 / 60; 今日显示为 XhYm。
- 待办完成标记: 若专注时有关联待办 (`currentTodoIndex != -1`), 将其 `isCompleted` 置为 `true`, 界面显示删除线与灰色文字。

4.3 单元测试 (设计思路)

采用手动测试进行项目检测, 详见第 5 节。

5 单元测试

5.1 测试案例定义

表 1: 手动测试案例

测试项	操作	预期结果
添加待办	输入名称“阅读”，时长 10 分钟	列表新增一项，显示“阅读 (10 分钟)”
空白待办	不输入名称直接确定	弹出警告“待办名称不能为空”
专注完成	点击待办“开始”，等待倒计时归零	弹窗提示完成，该待办变为灰色 + 删除线，总时长增加 10 分钟，图表更新
提前结束	专注中点击“提前结束”	已专注的时间被记录，窗口关闭
暂停/继续	专注中点击“暂停”，再点击“继续”	倒计时正确停止与恢复，按钮文字相应切换
快速开始	设定 5 分钟，点击快速开始按钮	专注正常计时，结束后不标记任何待办，总时长增加 5 分钟
音乐播放	目录下存在 <code>chuanyueshikong.mp3</code>	专注时循环播放，暂停时暂停，结束后停止
背景图设置	目录下存在 <code>mountain.jpg</code>	在开始专注时，背景即显示为此 jpg

5.2 测试结果

所有核心流程均通过，界面响应符合预期。

6 收获

6.1 信号与槽机制的应用

`FocusWindow` 作为独立窗口，通过信号将专注结果返回 `MainWindow`，实现模块解耦。例如，窗口内部倒计时结束后发射信号：

Listing 1: `FocusWindow` 完成信号发射

```
void FocusWindow::updateCountdown() {  
    if (remainingSeconds > 0) {  
        remainingSeconds--;  
        updateTimeDisplay();  
    }  
}
```

```
    } else {  
        timer->stop();  
        emit countdownFinished(totalSeconds);  
        // ...  
        close();  
    }  
}
```

主窗口中连接信号与槽：

```
connect(focusWindow, &FocusWindow::countdownFinished,  
        this, &MainWindow::onFocusFinished);
```

这种机制避免了窗口间直接调用，提高了代码的可维护性。

6.2 自定义控件与列表嵌入

通过继承 `QWidget` 实现 `TodoItemWidget`，并嵌入 `QListWidget`，实现了复杂的列表项布局。关键代码为：

```
QListWidgetItem *listItem = new QListWidgetItem(todoList);  
listItem->setSizeHint(itemWidget->sizeHint());  
todoList->setItemWidget(listItem, itemWidget);
```

此方式使得每个列表项可以包含多个控件，交互灵活。

6.3 资源管理体会

`QMediaPlayer` 在每次专注开始时动态创建，结束时手动 `delete`，但若窗口未关闭而重复启动会引发泄漏。后续应改进为复用或安全释放，加深了对 C++ 手动内存管理的重视。

7 AI 工具使用披露

- **AI 工具名称及版本：** DeepSeek（公开访问版本）
- **具体用途：** 代码 bug 问询与修复、界面布局调整、界面 ui（包括待办列表、坐标轴、按钮）美化、绘制柱状图
- **使用位置：**
 1. 待办列表 ui 实现（`TodoItemWidget` 的布局与样式）。
 2. 柱状图的绘制。
 3. 主窗口整体 ui 的布局设计（左右分区方案）；
 4. 部分按钮和控件的样式调整。
 5. 编写代码过程中的 bug 检测。